

Mindmap — ASCII only (`coding/`)

E/M/H = difficulty · ★ = must-nail · → = one-line hint

coding/data-structures/

advanced.md

coding/data-structures/advanced.md

└─ advanced.md

└─ Suffix Array

└─ Number of Distinct Substrings [M]

→ Build SA via prefix doubling ($O(n \log^2 n)$) or SA-IS ($O(n)$).
Build LCP via Kasai's $O(n)$ algorithm

└─ Longest Common Prefix of Suffixes (LCP Array) [M]

→ See implementation in count_distinct_substrings above. Longest repeated = $s[sa[idx] : sa[idx] + \max(lcp)]$

└─ LFU Cache – $O(1)$ Implementation

└─ LFU Cache [H]

→ `get(key)`: if key missing return -1. Increment `freq[key]`. Move key from `freq_to_keys[old_freq]` to `freq_to_keys[new_freq]`. Update `min_freq` if `old_freq == min_freq` and that bucket is now empty. `put(key, value)`: if key exists, update value and call get logic (increment freq). If capacity full, evict: call `popitem(last=False)` (LRU) from `freq_to_keys[min_freq]`, remove from all maps. Insert new key with `freq=1`. Set `min_freq=1` (new insertions always land at `freq 1`)

└─ Segment Tree

└─ Range Sum Query – Mutable (LC 307) [M]

→ Tree size $4*n$. build fills leaves and merges upward. update walks to the leaf, updates, merges on the way back. query recursively combines: if query fully covers current node, return stored sum; if no overlap, return 0; else recurse into both children

└─ Range Minimum Query (Segment Tree) [M]

→ Build, update, query are identical to range sum – swap + for min and 0 for inf in the no-overlap base case

└─ Count of Smaller Numbers After Self (LC 315) [M]

→ Coordinate compress first: sort unique values, assign ranks. Use BIT/segment tree of size m . Right-to-left pass: query prefix sum up to `rank[i]-1`, then increment `rank[i]`

└─ Number of Longest Increasing Subsequences (LC 673) [M]

→ For each `nums[i]` (left to right), query the tree over $[0, \text{rank}[i]-1]$ to get the best (len, cnt) for any element smaller than `nums[i]`. Then `new_len = len+1`, `new_cnt = cnt`. Point-update `rank[i]` with $(\text{new_len}, \text{new_cnt})$. Merge rule: keep the entry with larger length; if tie, add counts

└─ My Calendar I / II / III (LC 729 / 731 / 732) [M]

→ Use a sorted map (balanced BST) as a difference array: +1 at start, -1 at end. Scan prefix sums to find max overlap. Python uses `SortedList` from `sortedcontainers` or a `defaultdict` with sorted keys

└─ Binary Indexed Tree (Fenwick Tree)

└─ Range Sum Query – Mutable (BIT version) [M]

→ Store original array. On `update(i, val)`: compute `delta = val - nums[i]`, update `nums[i]`, then propagate delta through BIT.
`sumRange(l, r) = prefix(r+1) - prefix(l)`

└─ Count Inversions (Fenwick Tree) [M]

→ Sort unique values to get ranks. BIT stores counts. After inserting r , `query(r)` = count of elements $\leq r$ already inserted. Elements already inserted with rank $> r$ = `query(m) - query(r)` = inversions contributed by current element

└─ Reverse Pairs (LC 493) [H]

→ Coordinate-compress all values AND all $2*value$ together (to handle the $2*nums[j]$ query correctly). For each `nums[i]` (right to left): count elements with rank $\leq \text{rank of } (nums[i]-1) // 2$... Alternatively: use merge sort which is cleaner

└─ Number of Subarrays with Bounded Maximum (LC 795) [M]

→ Scan once; maintain `curr` = current run length of elements $\leq k$. When element $> k$, reset `curr` to 0. Accumulate `curr` into total

└─ Sparse Table (Extended)

└─ Sparse Table for Range Minimum Query (static) [M]

→ Build with two nested loops. Precompute $\log_2$ table to make queries $O(1)$ (no `math.log` call)

└─ Sparse Table RMQ with LCA Application [M]

→ DFS to build Euler tour array and `first[node]` = first occurrence index. Build sparse table on depths. `lca(u, v)`: query min-depth in $[first[u], first[v]]$, return that node

└─ Skip List (Full Implementation)

└─ Skip List Insert / Search / Delete [M]

→ `_find_predecessors(target)` traverses from the top level downward, collecting the rightmost node at each level whose value is $< target$. search checks level 0. add generates a random level, inserts node. erase removes one occurrence

└─ Monotonic Stack / Queue (Advanced)

└─ Maximum of Subarrays of Size K (Sliding Window Ma... [H]

→ For each new element: pop from back while `nums[back] ≤ nums[i]` (they're dominated). Push i . Pop from front if `front ≤ i - k` (out of window). When $i \geq k-1$, `nums[dq[0]]` is the answer

└─ Sum of Subarray Ranges (LC 2104) [H]

→ Two monotonic stack passes (one for max boundaries, one for min boundaries). Use strict vs. non-strict inequalities on one side to avoid double-counting duplicates

└─ Number of Submatrices That Sum to Target (LC 1074) [M]

→ Precompute 2D prefix sums. Outer two loops: fix $r1$ and $r2$. Inner loop: build running column sum, use `prefixSum - target` in a hash map to count subarrays

└─ Sqrt Decomposition

└─ Block Decomposition for Range Queries [M]

→ For `sumRange(l, r)`: if l and r are in the same block, iterate directly. Otherwise: sum partial left block, sum full middle blocks via `block_sum`, sum partial right block

array.md

coding/data-structures/array.md

└─ array.md

└─ Two Pointers

└─ Two Sum (sorted variant) [E]

→ Two Pointers on sorted array: $l=0$, $r=n-1$. If $\text{numbers}[l] + \text{numbers}[r] == \text{target}$, done. If $\text{sum} < \text{target}$, $l++$ (we need larger). If $\text{sum} > \text{target}$, $r--$ (we need smaller). The sorted invariant guarantees we never skip valid pairs

└─ 3Sum [M]

→ Fix + Two Pointers: Sort. For each i , if $\text{nums}[i] > 0$ break (sorted – no triplet can sum to 0). Skip duplicate i . Run two-pointer on the suffix. On match, skip duplicate left and right before advancing both. Three deduplication sites: i , left, right. Interview note: sorting is what makes the duplicate skipping and early break safe, so this pattern is usually the cleanest solution in interviews

└─ 3Sum Closest [M]

→ Sort + Two Pointers with closest tracking: Sort. For each i , two-pointer on suffix. Compute $s = \text{nums}[i] + \text{nums}[l] + \text{nums}[r]$. Update closest if $|s - \text{target}| < |\text{closest} - \text{target}|$. If $s < \text{target}$, $l++$. If $s > \text{target}$, $r--$. If exact match, return immediately

└─ 4Sum [M]

→ Two nested loops + Two Pointers: Sort. Outer loop i , inner loop $j = i+1$. Skip duplicate i and j . Two pointers $l=j+1$, $r=n-1$. Same pointer logic as 3Sum. Early termination: if the smallest possible sum for the current i is already too large, break; if the largest possible sum is still too small, continue

└─ Container with Most Water [M]

→ Two Pointers – advance the shorter line: $l=0$, $r=n-1$. Compute area. Always advance the pointer pointing to the shorter line – moving the taller line can never increase $\min(h[l], h[r])$ while width also decreases

└─ Trapping Rain Water [H]

→ Two Pointers – binding constraint side: $l=0$, $r=n-1$, $l_{\max}=r_{\max}=0$. If $l_{\max} \leq r_{\max}$, the left side is the constraint, so water at l is $l_{\max} - \text{height}[l]$ and we advance l . Otherwise, the right side is the constraint, so water at r is $r_{\max} - \text{height}[r]$ and we decrement r

└─ Remove Duplicates from Sorted Array [M]

→ Two Pointers – slow/fast write pattern: $\text{slow}=1$. For fast in $1..n-1$: if $\text{nums}[\text{fast}] \neq \text{nums}[\text{slow}-1]$, write $\text{nums}[\text{slow}] = \text{nums}[\text{fast}]$, $\text{slow}++$. Return slow

└─ Next Permutation [M]

→ Three-step: find rightmost descent, swap, reverse suffix: 1. Scan right-to-left to find index i where $\text{nums}[i] < \text{nums}[i+1]$ (rightmost ascending pair from the right). 2. If none found, the whole array is descending – just reverse it. 3. From the right, find the smallest element greater than $\text{nums}[i]$; swap them. 4. Reverse $\text{nums}[i+1:]$ to get the smallest permutation of the suffix

└─ Sliding Window

└─ Max Consecutive Ones III [M]

→ Variable sliding window – zero count tracking: Expand r . If

$\text{nums}[r] == 0$, increment zero count. If $\text{zeros} > k$, shrink l until $\text{zeros} \leq k$ (moving l past a zero decrements zero count). Track $r - l + 1$ as window size

└─ Longest Repeating Character Replacement [M]

→ Variable window with monotone max_freq trick: Expand r . Update freq map. If $(r - l + 1) - \text{max_freq} > k$, shrink l by one (decrement freq of $s[l]$). Key insight: max_freq never needs to decrease – a smaller max_freq cannot yield a larger valid window

└─ Subarrays with K Different Integers [M]

→ Exactly- $k = \text{at_most}(k) - \text{at_most}(k-1)$: $\text{at_most}(k)$ counts subarrays with $\leq k$ distinct integers. Use standard sliding window: expand right, if distinct count $> k$ shrink left, add $r - l + 1$ (all subarrays ending at r and starting at $l..r$). Answer = $\text{at_most}(k) - \text{at_most}(k-1)$

└─ Minimum Window Substring [H]

→ Sliding window with have counter: Build need freq map for t . Expand r . For each char, if its count in window hits required count, $\text{have}++$. When $\text{have} == \text{len}(\text{need})$, try to shrink: record window, shrink l . If removing $s[l]$ drops a count below required, $\text{have}--$

└─ Sliding Window Maximum [H]

→ Monotonic decreasing deque: For each r : pop rear of deque while deque is non-empty and $\text{nums}[\text{deque}[-1]] \leq \text{nums}[r]$. Append r . Pop front if $\text{deque}[0] \leq r - k$ (out of window). Once $r \geq k-1$, the front index gives the current window max

└─ Prefix Sum

└─ Subarray Sum Equals K [M]

→ Prefix sum + hash map complement lookup: Scan left to right, maintaining running_sum and a hash map seen of count of each prefix sum seen so far. Seed $\text{seen}[0] = 1$ (empty prefix). For each element, $\text{count} += \text{seen}[\text{running_sum} - k]$

└─ Subarray Sums Divisible by K [M]

→ Prefix modulo – same remainder pairs: Scan with $\text{running \% k}$. Handle negative modulo: $(\text{running \% k} + k) \% k$. Seed $\text{seen}[0] = 1$. For each position, $\text{count} += \text{seen}[\text{running \% k}]$ before incrementing

└─ Continuous Subarray Sum [M]

→ Prefix modulo – first-occurrence index map: Seed $\text{seen}[0] = -1$ (empty prefix at index -1). For each index i , compute rem . If rem in seen and $i - \text{seen}[\text{rem}] \geq 2$, return True. Otherwise, record $\text{rem} \rightarrow i$ only if not already present (want first occurrence)

└─ Product of Array Except Self [M]

→ Two-pass left/right product accumulation: Pass 1: $\text{output}[i] = \text{product of nums}[0..i-1]$. Pass 2: maintain $\text{right} = 1$, scan right to left, multiply $\text{output}[i] *= \text{right}$, then $\text{right} *= \text{nums}[i]$

└─ Kadane's Algorithm

└─ Maximum Subarray [E]

→ Kadane's algorithm – local reset on negative prefix: Initialize $\text{cur} = \text{best} = \text{nums}[0]$ (handles all-negative case). For each subsequent element: $\text{cur} = \max(x, \text{cur} + x)$. Update $\text{best} = \max(\text{best}, \text{cur})$

└─ Maximum Product Subarray [M]

→ Track max and min simultaneously: At each element x : new $\text{max_prod} = \max(x, \text{max_prod} * x, \text{min_prod} * x)$, new $\text{min_prod} = \min(x, \text{max_prod} * x, \text{min_prod} * x)$. Use temps to avoid overwriting. Update global best

- Maximum Circular Subarray Sum (Maximum Sum Circul... [M]
 - Kadane's max + Kadane's min on total sum: Edge case: if all elements are negative, total - kadane_min = 0 (the empty subarray), which is wrong. In this case, return kadane_max
 - Dutch National Flag / Partitioning
 - Sort Colors [M]
 - Dutch National Flag - three-pointer partition: lo=0, mid=0, hi=n-1. While mid <= hi: if nums[mid]==0, swap with lo, advance both; if nums[mid]==1, advance mid; if nums[mid]==2, swap with hi, decrement hi - do NOT advance mid (swapped element from hi is unexamined)
 - Find All Numbers Disappeared in an Array [M]
 - Sign-flip in-place visited marking: For each value x = abs(nums[i]), negate nums[x-1]. After marking, collect all indices where value is still positive - those indices +1 are the missing values
 - Boyer-Moore Voting
 - Majority Element [M]
 - Boyer-Moore voting - cancellation argument: This works because: imagine each "non-candidate" vote cancels one "candidate" vote. Since majority has > n/2 votes, it survives after all cancellations
 - Majority Element II [M]
 - Boyer-Moore with two candidates: Maintain c1, c2, cnt1, cnt2. For each x: if x == c1, cnt1++; elif x == c2, cnt2++; elif cnt1 == 0, c1=x, cnt1=1; elif cnt2 == 0, c2=x, cnt2=1; else cnt1--, cnt2--. Second pass: count actual frequencies and filter > n/3
 - Floyd's Cycle Detection (on Arrays)
 - Find the Duplicate Number [M]
 - Floyd's cycle detection on implicit linked list: Phase 1 - slow = nums[slow], fast = nums[nums[fast]] until they meet (inside cycle). Phase 2 - reset slow = nums[0] (the start), advance both at speed 1; they meet at the cycle entry = duplicate
 - Difference Array
 - Difference Array Pattern [M]
 - Diff array: +v at l, -v at r+1; prefix sum reconstructs range adds
 - Car Pooling [M]
 - Build diff[0..1001]. For each trip: diff[from] += num; diff[to] -= num. Reconstruct prefix sum; if any prefix sum > capacity, return False
 - Corporate Flight Bookings [M]
 - Build diff[0..n+1]. For each booking: diff[first] += seats; diff[last+1] -= seats. Prefix sum of diff[1..n] is the answer
 - Range Addition [M]
 - Build diff[0..n] (size n+1 to handle r+1 = n). Apply all updates. Prefix-sum diff[0..n-1] in-place
 - Miscellaneous Array Techniques
 - Best Time to Buy and Sell Stock [M]
 - min_price = inf, max_profit = 0. For each price: min_price = min(min_price, price), max_profit = max(max_profit, price - min_price)
 - Move Zeroes [M]
 - slow = 0. For each fast: if nums[fast] != 0, set nums[slow] = nums[fast], slow++. After the loop, zero out nums[slow..n-1]
- Two Sum (hash map variant) [E]
 - Hash map complement lookup: For each (i, x): compute complement = target - x. If in map, return [map[complement], i]. Else record x → i. Checking before recording ensures we don't use same index twice
- Jump Game II [M]
 - Greedy BFS - level-by-level farthest reach: Advance i from 0 to n-2. Update farthest = max(farthest, i + nums[i]). When i == current_end: a new jump is needed, jumps++, current_end = farthest. Stop when current_end >= n-1
- Spiral Matrix [M]
 - Boundary shrinking - four-direction cycling: While top <= bottom and left <= right: traverse right along top row, top++; traverse down along right col, right--; if top <= bottom, traverse left along bottom row, bottom--; if left <= right, traverse up along left col, left++. The inner checks prevent double-counting for single row/col cases
- Set Matrix Zeroes [M]
 - Use first row/col as markers - O(1) space: Record if row 0 or col 0 should be zeroed (check for existing zeros). For all other cells, if matrix[i][j] == 0, set matrix[i][0] = 0 and matrix[0][j] = 0. Then zero out rows and cols using those markers. Finally handle row 0 and col 0 separately
- Longest Consecutive Sequence [M]
 - Hash set - start sequences only from minimums: Insert all values into a set. For each n, if n-1 not in set (start of sequence), count consecutive n, n+1, n+2, ... until gap. Update best length
- Rotate Array [M]
 - Triple reversal trick: Normalize k = k % n. Reverse nums[0:n]. Reverse nums[0:k]. Reverse nums[k:n]
- Median of Two Sorted Arrays [H]
 - Binary search on shorter array partition: Binary search on shorter array (WLOG m <= n). Partition nums1 at i, derive nums2 partition j = half - i. Check: nums1[i-1] <= nums2[j] and nums2[j-1] <= nums1[i]. Adjust binary search accordingly. For odd total, median = max(left sides). For even, median = (max(left) + min(right)) / 2
- Range / Immutable Prefix Queries
 - Range Sum Query - Immutable (LC 303) [M]
 - Prefix sum array - O(1) per query: Precompute prefix[0..n] where prefix[0] = 0 and prefix[i] = prefix[i-1] + nums[i-1]. Each query: return prefix[right+1] - prefix[left]
 - Range Sum Query 2D - Immutable (LC 304) [M]
 - 2D prefix sum (inclusion-exclusion): sum(r1,c1,r2,c2) = prefix[r2+1][c2+1] - prefix[r1][c2+1] - prefix[r2+1][c1] + prefix[r1][c1]
 - Contiguous Array (LC 525) [M]
 - Prefix sum with 0→-1 transform + first-occurrence hash map: Seed seen = {0: -1}. For each index i, update running. If running in seen, candidate length = i - seen[running]. Else store seen[running] = i. Never overwrite (want earliest occurrence for max length)
- Two-pass / Greedy
 - Jump Game (LC 55) [M]
 - Greedy - track farthest reachable index: reach = 0. For each i

- in $0..n-1$: if $i > \text{reach}$, return False. Update $\text{reach} = \max(\text{reach}, i + \text{nums}[i])$. If $\text{reach} \geq n-1$ at any point, return True
- Candy (LC 135) [M]
 - Two-pass greedy – left then right: Initialize all to 1. Left pass enforces left-rising constraint. Right pass enforces right-rising constraint using max to preserve the larger requirement
- Gas Station (LC 134) [M]
 - Greedy – reset start on deficit: $\text{total} = 0$, $\text{tank} = 0$, $\text{start} = 0$. For each i : $\text{tank} += \text{gas}[i] - \text{cost}[i]$, $\text{total} += \text{gas}[i] - \text{cost}[i]$. If $\text{tank} < 0$, set $\text{start} = i + 1$, reset $\text{tank} = 0$. Return start if $\text{total} \geq 0$ else -1
- Matrix
 - Rotate Image (LC 48) [M]
 - Transpose then reverse each row: Transpose: for i in $\text{range}(n)$: for j in $\text{range}(i+1, n)$: swap $\text{matrix}[i][j]$ and $\text{matrix}[j][i]$. Reverse: for row in matrix : $\text{row.reverse}()$
 - Search a 2D Matrix (LC 74) [M]
 - Binary search treating the matrix as a flat sorted array: $\text{lo}=0$, $\text{hi}=m*n-1$. At each mid: $\text{val} = \text{matrix}[\text{mid} // n][\text{mid} \% n]$. Compare with target; adjust lo/hi accordingly
- Intervals
 - Merge Intervals (LC 56) [M]
 - Sort by start, linear merge scan: Sort by start. Initialize $\text{merged} = [\text{intervals}[0]]$. For each subsequent interval: if $\text{interval.start} \leq \text{merged}[-1].\text{end}$, merge by updating $\text{merged}[-1].\text{end} = \max(\text{merged}[-1].\text{end}, \text{interval.end})$. Else append
 - Insert Interval (LC 57) [M]
 - Three-phase linear scan: before, overlap, after: Phase 1: while $\text{intervals}[i].\text{end} < \text{new.start}$, append. Phase 2: while $\text{intervals}[i].\text{start} \leq \text{new.end}$, extend $\text{new.start} = \min(\text{new.start}, \dots)$ and $\text{new.end} = \max(\text{new.end}, \dots)$. Append merged. Phase 3: append remaining
 - Non-overlapping Intervals (LC 435) [M]
 - Greedy – sort by end, keep earliest-ending non-conflicting interval: Sort by end. $\text{prev_end} = -\text{inf}$, $\text{kept} = 0$. For each interval: if $\text{start} \geq \text{prev_end}$, keep it ($\text{kept}++$, update $\text{prev_end} = \text{end}$). Else skip (remove it). Answer = $n - \text{kept}$
- Miscellaneous (Continued)
 - First Missing Positive (LC 41) [M]
 - Index-as-hash – cyclic placement of values in range $[1, n]$: Swap phase: for each i , while $1 \leq \text{nums}[i] \leq n$ and $\text{nums}[\text{nums}[i]-1] \neq \text{nums}[i]$, swap $\text{nums}[i]$ with $\text{nums}[\text{nums}[i]-1]$. Scan phase: return first $i+1$ where $\text{nums}[i] \neq i+1$, else return $n+1$
- Prefix Sum
 - Subarray Sum Equals K (with negative numbers) [M]
 - Initialize map with $\{0: 1\}$ (empty prefix). Walk the array accumulating running_sum ; add $\text{count_map}[\text{running_sum} - k]$ to the answer; then increment $\text{count_map}[\text{running_sum}]$
 - Contiguous Array (Equal 0s and 1s) [M]
 - Initialize $\{0: -1\}$. For each index, compute prefix sum (treating 0 as -1). If seen before, update $\text{max_len} = \max(\text{max_len}, i - \text{first_seen}[\text{prefix}])$. Otherwise, store $\text{first_seen}[\text{prefix}] = i$
 - Product of Array Except Self (no division) [M]
 - $\text{output}[i]$ after left pass = product of $\text{nums}[0..i-1]$. Then walk

right to left with a `right_product` variable, multiply $\text{output}[i] *= \text{right_product}$, then $\text{right_product} *= \text{nums}[i]$

graph.md

coding/data-structures/graph.md

└─ graph.md

└─ BFS on Graphs

└─ Rotting Oranges [M]

→ Multi-source BFS – simultaneous spread from all rotten sources: Multi-source start avoids $O(R \times M \times N)$ repeated BFS. Return `max_time` if fresh == 0, else -1

└─ Number of Islands (BFS) [M]

→ Seed the queue with the trigger cell; mark visited on enqueue (not dequeue) to prevent duplicate entries. Mutating the grid avoids an extra visited array

└─ Walls and Gates (LC 286) [M]

→ Only enqueue cells that are INF – this acts as the visited guard. The first time a room is reached is always via its nearest gate

└─ 01 Matrix (LC 542) [M]

→ Initialize dist matrix with 0 for zeroes, INF for ones. Enqueue all zeroes at start. Only update a cell if current dist > neighbor dist + 1

└─ Word Ladder [H]

→ BFS on implicit word graph – $L \times 26$ mutation enumeration: For each word dequeued, try all single-char mutations; if mutation == endWord, return. Otherwise enqueue if the word is still in the set and then delete it

└─ Shortest Path in Binary Matrix [M]

→ BFS from (0,0) – 8-directional, mark on enqueue: Mark cells visited by setting to 1 as you enqueue (not after dequeue) to prevent duplicate enqueueing; return distance when (n-1, n-1) is dequeued

└─ Minimum Knight Moves [M]

→ BFS with symmetry reduction to first quadrant: Use a visited set; use abs values to exploit symmetry; allow coordinates down to -2 (buffer for (0,0)/(1,1) edge cases)

└─ Employee Importance [M]

→ Hash map + BFS over subordinate ids: Build {id: employee} map; BFS – dequeue id, add importance, enqueue all subordinate ids

└─ Find if Path Exists in a Graph [M]

→ Union-Find – reachability in $O(E \alpha(N))$: Path compression + union by rank for optimal performance

└─ Find Center of Star Graph [M]

→ $O(1)$ – center appears in both the first and second edges: Check if edges[0][0] is in edges[1]; if so, it's the center; otherwise edges[0][1] is the center

└─ DFS on Graphs

└─ Number of Islands [M]

→ DFS sinking – flood-fill each component, count triggers: Sinking avoids a separate visited array – the mutation is the visit mark. Increment count only on the initial call, not within DFS

└─ Flood Fill [M]

→ DFS recolor – guard against same-color infinite loop: Guard with if original == color: return image before DFS

└─ Max Area of Island [M]

→ DFS returning component size – sink inline: `max(dfs(r,c) for all r,c)` – zero-cost for water cells

└─ Surrounded Regions [M]

→ Reverse DFS – mark border-safe '0's, then flip interior: Walk all 4 borders, DFS from each '0' found there

└─ Pacific Atlantic Water Flow [M]

→ Reverse BFS from both ocean borders – intersect reachable sets: Reverse BFS condition – expand to neighbors with height $\geq$ current height (uphill in the reverse direction)

└─ All Paths From Source to Target [M]

→ DFS backtracking on DAG – no visited set needed: `path.append(nei)`, `recurse`, `path.pop()`

└─ Number of Provinces (LC 547) [M]

→ Use a visited array instead of mutating the matrix. The matrix is symmetric but you only need to follow one direction per city

└─ Number of Enclaves (LC 1020) [M]

→ Walk all 4 borders; DFS from each '1' encountered, sinking to 0. After traversal, sum remaining 1-cells

└─ Clone Graph [M]

→ DFS with original-clone map – register before recursing: Guard if n in visited: return visited[n] handles cycles. Register BEFORE recursing into neighbors

└─ Find Eventual Safe States [M]

→ Three-color DFS – 0=unvisited, 1=in-progress, 2=safe: Mark gray before recursing neighbors; mark black after all neighbors are confirmed safe

└─ Topological Sort

└─ Course Schedule [M]

→ Kahn's BFS topo sort – cycle detection via leftover nodes: If completed == numCourses, no cycle

└─ Course Schedule II [M]

→ Kahn's topo sort – collect removal order: Append node to order as it's dequeued; return order or []

└─ Alien Dictionary [H]

→ Edge extraction from adjacent word pairs + Kahn's topo sort: Invalid input detection – if word A is a prefix of word B but A appears after B (e.g., "abc" before "ab"), return "" immediately

└─ Minimum Number of Vertices to Reach All Nodes [M]

→ Nodes with in-degree 0 – the only possible starting set: Collect all destination nodes from edges – these have in-degree ≥ 1 ; return all nodes not in this set

└─ Shortest Path / Weighted

└─ Network Delay Time (Dijkstra's) [M]

→ Dijkstra's – min-heap SSSP with stale-entry skip: After Dijkstra, answer = max of all shortest distances; -1 if any node unreachable

└─ Swim in Rising Water [M]

→ Modified Dijkstra – minimize maximum edge weight (bottleneck path): The first time we reach (n-1,n-1), we have the answer

└─ Cheapest Flights Within K Stops [M]

→ Bellman-Ford with k+1 rounds – copy dist array each round: Copy dist array each round to prevent using edges discovered in the same round (would allow more than 1 edge per round effectively)

└─ Path with Minimum Effort (LC 1631) [M]

→ First pop of (rows-1, cols-1) from the heap is the answer. Mark

- visited on pop to avoid reprocessing
- Shortest Path in a DAG [M]
 - Initialize distances, process nodes in topo order, and update $\text{dist}[v] = \min(\text{dist}[v], \text{dist}[u] + w)$ for each edge
- Bipartite / Coloring
 - Is Graph Bipartite? [M]
 - BFS 2-coloring – alternating colors, fail on same-color neighbor: Initialize all colors to -1 (uncolored). For each unvisited node, BFS assigning color 0; assign 1 – color[node] to unvisited neighbors; return False if neighbor has same color
 - Possible Bipartition (LC 886) [M]
 - People labeled 1..n – initialize color array of size n+1. For each uncolored node, BFS alternating colors; return False if same-color conflict found
- Advanced
 - Redundant Connection [M]
 - Union-Find – first edge connecting already-connected nodes is redundant: For each edge (a, b): if union(a, b) returns False (same component), return [a, b]
 - Minimum Spanning Tree (Kruskal's) [M]
 - Kruskal's – sort edges by weight, greedily add if no cycle: Sort edges by weight; for each edge, union its endpoints if they're in different components; stop after adding V-1 edges
 - Longest Path in a DAG [M]
 - Kahn's topo sort + DP – relax $\text{dp}[v] = \max(\text{dp}[u] + 1)$; $\text{dp}[\text{node}] = \max(\text{dp}[\text{node}], \text{dp}[\text{prev}] + 1)$ as edges are relaxed during Kahn's
 - Reconstruct Itinerary (LC 332) [M]
 - Use a stack-based iterative post-order DFS: push node to result when its adjacency list is exhausted; reverse at the end
 - Critical Connections / Bridges (LC 1192) [M]
 - Track parent to avoid treating the tree edge back to parent as a back-edge. Update $\text{low}[u] = \min(\text{low}[u], \text{low}[v])$ after recursing into v; $\text{low}[u] = \min(\text{low}[u], \text{disc}[v])$ for back-edges
 - Minimum Height Trees (LC 310) [M]
 - Decrement n by the number of leaves removed each round; stop when $n \leq 2$ – remaining nodes are the answer
- Advanced Graph Algorithms
 - Strongly Connected Components – Kosaraju's Algorithm [M]
 - Build adjacency list and its transpose. DFS on original, recording finish order in a stack. Then repeatedly pop from the stack and DFS on the transposed graph – all reachable unvisited nodes form one SCC
 - Minimum Spanning Tree – Prim's Algorithm [M]
 - visited set tracks MST nodes. Pop from heap; if already visited, skip. Otherwise mark visited, add weight to MST cost, push all unvisited neighbours into the heap
 - All-Pairs Shortest Path – Floyd-Warshall [M]
 - Initialize $\text{dist}[i][j]$ to edge weight if edge exists, 0 if $i == j$, infinity otherwise. Triple loop: for each k, for each i, for each j: $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$

hashing.md

coding/data-structures/hashing.md

- hashing.md
 - Complement Map
 - Two Sum [E]
 - Single pass. Before storing x, look up target - x. If found, return [seen[complement], i]. Store x → i after checking to avoid using the same index twice
 - 3Sum (hash-based) [M]
 - Sort. Skip duplicate values of a. For the inner scan, use a seen set: if target - b in seen, record triplet; else add b to seen
 - 4Sum (hash-based) [M]
 - Fix nums[i] and nums[j]. Inner target is target - nums[i] - nums[j]. Use seen set for two-sum on remaining elements
 - Max Number of K-Sum Pairs [M]
 - Build Counter. For each unique x, pairs formed = $\min(\text{freq}[x], \text{freq}[k - x])$ if $x \neq k - x$, else $\text{freq}[x] // 2$. Sum all
 - Frequency Map
 - Valid Anagram [E]
 - Counter(s) == Counter(t). Or use a single 26-element array: increment for s, decrement for t, check all zeros
 - First Unique Character in a String [M]
 - Counter in one pass; second pass finds first with count 1. Two O(n) passes
 - Ransom Note [M]
 - Use a hash map or Counter; if any needed character drops below zero, return false
 - Group Anagrams [M]
 - For each string, compute tuple(sorted(s)) as key, append to defaultdict(list)
 - Find All Anagrams in a String [M]
 - Use two Counter maps (window and p). Track have = number of chars where window count equals p count. When have == len(p_count), record the start
 - Top K Frequent Elements [M]
 - Counter → buckets list of size n+1 where buckets[f] holds all numbers with frequency f. Iterate from index n down and collect until we have k elements
 - Subdomain Visit Count [M]
 - For "9 discuss.leetcode.com" add 9 to discuss.leetcode.com, leetcode.com, and com. Format output as "count domain" strings
 - Sort Characters by Frequency [M]
 - Counter, then sort by count descending, rebuild via $\text{ch} * \text{count}$ concatenation
 - Minimum Window Substring [H]
 - Expand right, update have when a char's count first meets requirement. Shrink left while have == need. Record minimum window during each valid state
 - Prefix Sum + Map
 - Longest Subarray with Sum K [M]
 - first_seen = {0: -1}. At index i, if prefix - k in map, update best = max(best, i - first_seen[prefix - k]). Only insert prefix if not already present (preserve earliest index)

- Count Number of Nice Subarrays [M]
 - seen = {0: 1}. Running sum increments by 1 for odd elements, 0 for even. Look up prefix - k in seen before updating map
- Binary Subarrays with Sum [M]
 - Identical to subarray sum equals k. The binary constraint doesn't change the algorithm, only guarantees prefix is non-negative
- Subarray Sum Equals K [M]
 - Maintain running prefix sum. Before updating the map, check seen[prefix - k]. Initialize seen = {0: 1} to handle subarrays starting at index 0
- Contiguous Array (Max Equal 0/1 Subarray) [M]
 - Track running sum with 0→-1 transform. When prefix repeats, the subarray between the two occurrences has sum 0. Store first_seen = {0: -1} and compare i - first_seen[prefix]
- Make the Array Sum Divisible by P [M]
 - rem = sum(nums) % P. If rem == 0, return 0. Use seen = {0: -1}. At each step store prefix % P → i. Look up (prefix - rem) % P
- Subarray Sums Divisible by K [M]
 - seen = {0: 1}. For each element, compute prefix % K (handle negatives: % K in Python already returns non-negative). Add seen[(prefix % K)] to count. Increment map
- Design
 - LRU Cache [M]
 - On get: look up node, move to front, return val. On put: if key exists update and move to front; else insert at front; if over capacity evict tail
 - Insert Delete GetRandom O(1) [M]
 - Insert appends to list and stores index in map. Remove swaps target with last element, updates map for the moved element, pops the list, deletes map entry for removed value
 - Design HashMap [M]
 - put scans bucket for existing key (update) or appends. get scans for key, returns -1 if absent. remove filters out the key
- Set Operations
 - Contains Duplicate [M]
 - Single pass: if x in seen return True; else seen.add(x). Short-circuits on first duplicate
 - Intersection of Two Arrays [M]
 - set(nums1) & set(nums2) in Python. For an explicit approach: iterate the smaller set, check membership in the larger
 - Two Sum Less Than K [E]
 - After sort, l = 0, r = n - 1. Converge inward. Track best = max(best, sum) when sum < k
- Miscellaneous
 - Longest Consecutive Sequence [M]
 - Build set. For each x, if x - 1 not in set, extend the chain x, x+1, x+2, ... while each successor is in the set
 - Substring with Concatenation of All Words [M]
 - For each offset in [0, word_len), maintain a sliding window of exactly num_words * word_len characters. Add/remove one word at a time from window ends
 - Max Points on a Line [M]
 - For each anchor i, build a slope map. Use gcd to normalize: slope = (dy // g, dx // g). Handle vertical lines (dx == 0) and

- same-point duplicates separately
- Rolling Hash / Dedup
 - Longest Duplicate Substring (Rabin-Karp) [M]
 - Hash = sum(ord(s[i]) * base^(L-1-i)) % mod for window. Rolling update: new_hash = (old_hash * base - ord(left) * base^L + ord(right)) % mod. Store hashes in a set. Return the window on collision
 - Find Duplicate File in System [M]
 - For each entry split on spaces: first token is directory, rest are name(content) tokens. Extract content between (and). Map content → list[full_path]. Return groups with size ≥ 2
 - 4Sum II [M]
 - Two nested loops for AB → Counter. Two nested loops for CD → look up complement. Sum all matching counts
- Prefix Sum + Hashing
 - Subarray Sums Divisible by K (LC 974) [M]
 - Initialize {0: 1}. For each element, compute remainder = running_sum % k. In Python, % always returns non-negative values, so no adjustment needed. Add count_map[remainder] to the answer, then increment count_map[remainder]
 - Contiguous Array (LC 525) [M]
 - Initialize {0: -1}. For each index i, update prefix. If prefix is in the map, update max_len = max(max_len, i - first_seen[prefix]). Otherwise record first_seen[prefix] = i
 - 4Sum II (LC 454) [M]
 - ab_count = Counter(a + b for a in A for b in B). Then total = sum(ab_count[-(c + d)] for c in C for d in D)

heap.md

coding/data-structures/heap.md

└─ heap.md

Top-K Pattern

Kth Largest Element in a Stream [M]

→ On each add, push the new value. If heap size exceeds k, pop the minimum. Root is the answer in $O(1)$; each insert is $O(\log k)$

Last Stone Weight [M]

→ Pop twice, push $\text{abs}(a - b)$ if non-zero. Repeat until one or zero stones remain

K Closest Points to Origin [M]

→ For each point compute $x^2 + y^2$. Push $(-\text{dist}, x, y)$ onto a max-heap. If size exceeds k, pop the farthest. Remaining heap contains the k closest

Top K Frequent Elements [M]

→ Build Counter in $O(n)$. Heap push $(\text{freq}, \text{num})$ for each unique num; pop when size $> k$. Alternatively, bucket sort by frequency for $O(n)$

Furthest Building You Can Reach [M]

→ For each upward jump, assign a ladder (push jump to heap). If ladders exhausted, pop the smallest ladder-jump, reclaim it as bricks. If bricks insufficient for the current jump, stop

Kth Largest Element in an Array [M]

→ For each number, push to heap. If heap exceeds size k, pop the minimum. Root after full pass is the answer in $O(1)$

Top K Frequent Words [M]

→ Count with Counter, then ask for the k best items under that custom key. This is safer than hand-rolling a size-k heap because naive tuple ordering is easy to get wrong for ties

Scheduling / Reorganization

Task Scheduler [M]

→ Count frequencies. Compute max_freq. The formula models "frames" of size $n+1$ with the most frequent task anchoring each frame. Return $\text{max}(\text{formula}, \text{len}(\text{tasks}))$

Reorganize String [M]

→ Impossible if $\text{max_freq} > (\text{len}(s) + 1) // 2$. Otherwise, greedily fill from the heap

Two Heaps

Find Median from Data Stream [M]

→ Always push to lo, then move lo's max to hi to maintain order. Rebalance sizes so lo is never smaller than hi. Median is either $\text{lo}[0]$ or the average of both tops

Sliding Window Median [H]

→ Maintain lo (max-heap) and hi (min-heap). Slide window: add new element, mark removed element as "invalid". Rebalance heaps. When reading tops, skip invalid elements

IPO (Maximize Capital) [M]

→ Sort $\text{zip}(\text{capital}, \text{profits})$ by capital. Use a pointer i advancing when $\text{projects}[i][0] \leq w$. Max-heap holds unlocked profits (negated). Repeat k times

K-Way Merge

Merge K Sorted Lists [H]

→ Seed heap with head of each non-null list. Use list_id as

tie-breaker to avoid comparing ListNode objects (not comparable in Python)

Find K Pairs with Smallest Sums [M]

→ Pop smallest (sum, i, j) , record pair. Push $(\text{nums1}[i] + \text{nums2}[j+1], i, j+1)$ if $j+1 < \text{len}(\text{nums2})$. Stop after k pops

Kth Smallest Element in a Sorted Matrix [M]

→ Push $(\text{matrix}[i][0], i, 0)$ for all i. Pop k-1 times advancing $(\text{matrix}[i][j+1], i, j+1)$. The k-th pop is the answer

Smallest Range Covering Elements from K Lists (He... [M]

→ Seed heap with $(\text{lists}[i][0], i, 0)$. Track $\text{cur_max} = \text{max}$ of all initial first elements. Each pop gives a new candidate min; update range if $\text{cur_max} - \text{min}$ is smaller. Stop when any list is exhausted

K-th Smallest in M Sorted Arrays [M]

→ Push $(\text{arrays}[i][0], i, 0)$ for all i. Pop and push $(\text{arrays}[i][j+1], i, j+1)$ until k pops done

Maximum CPU Load [M]

→ Sort jobs by start. For each job, pop from heap all jobs with $\text{end} \leq \text{job.start}$. Push current job's end time and load. Track running sum of active loads and record maximum

Dijkstra / Graph

Network Delay Time [M]

→ Build adjacency list. Push $(0, k)$. Pop min dist node; skip if already visited. Relax neighbors. Track visited set. Answer = $\text{max}(\text{dist.values}())$ if $\text{len}(\text{dist}) == n$ else -1

Path with Minimum Effort [M]

→ Push $(0, 0, 0)$. For each pop, update neighbors with $\text{max}(\text{effort}, \text{abs diff})$. Skip if already visited at a better effort

Swim in Rising Water [M]

→ Push $(\text{grid}[0][0], 0, 0)$. Pop min elevation; if it's the destination return it. Mark visited. Push unvisited neighbors with $\text{max}(\text{current_t}, \text{grid}[\text{nr}][\text{nc}])$

Design

Design Twitter [M]

→ Global time counter increments with each tweet. Each user has a list of $(\text{time}, \text{tweetId})$. For feed: seed heap with latest tweet from each followee+self. Pop max; push that user's next tweet. Collect 10

Ugly Number II [M]

→ Push 1. Pop min (= current ugly). Push $\text{val} * 2, \text{val} * 3, \text{val} * 5$ if not seen. Repeat n times

Heap Applications

Reorganize String (LC 767) [M]

→ Alternate approach (cleaner): pop the top character, append it, push the previous character back (if count > 0). This naturally avoids placing the same character twice in a row

IPO - Maximize Capital (LC 502) [M]

→ Sort $(\text{capital}, \text{profit})$ pairs. Pointer i advances while $\text{capital}[i] \leq w$. After unlocking, pop from the max-heap and add profit to w. Repeat k times

Minimum Refueling Stops (LC 871) [M]

→ Walk through stations in order. While $\text{fuel} < \text{station.position} - \text{current_position}$ and heap is non-empty, pop the largest fuel and add it to fuel (increment stops). If still can't reach the next station, return -1

linked-list.md

coding/data-structures/linked-list.md

└─ linked-list.md

└─ In-Place Reversal

└─ Reverse Linked List [E] ★

→ Three-pointer iterative reversal: Save `nxt = curr.next` before overwriting. Set `curr.next = prev`. Advance `prev = curr`, `curr = nxt`. When `curr` is `None`, `prev` is the new head. Watch the order carefully: save next before rewiring or you lose the rest of the list

└─ Reverse Linked List II [E]

→ Find pre-node + in-place splice-reversal: Each iteration: save `nxt = curr.next`, detach `nxt` from its position, reattach it after `pre`. This inserts nodes one by one at the front of the reversed section. After right – left iterations, the segment is reversed. If `left == right`, the loop runs zero times and the list stays unchanged

└─ Palindrome Linked List [M]

→ Find middle + reverse second half + compare: Slow/fast to find middle. If the list has odd length, advance slow one more step to skip the middle node. Reverse from that point onward. Walk two pointers – one from head, one from reversed head – checking values. Restore (optional): reverse second half back

└─ Reorder List [M] ★

→ Find middle + reverse second half + interleave: Slow/fast to find mid. Reverse second half. Merge two halves alternating: take one from first, one from second (reversed), repeat until second half is exhausted

└─ Reverse Nodes in K-Group [M]

→ Count-verify + in-place reversal per group: Check-count loop + standard in-place reversal of exactly `K` nodes. After reversing, head (original) is the tail of the reversed group; link it to the recursive result

└─ Fast / Slow Pointers

└─ Linked List Cycle [M]

→ Floyd's fast/slow pointer cycle detection: Start both at head. Loop: `slow = slow.next`, `fast = fast.next.next`. Check `slow` is `fast` (identity, not equality). If `fast` or `fast.next` is `None`, exit – no cycle. The identity check matters whenever node values can repeat

└─ Middle of the Linked List [M]

→ Fast/slow one-pass middle finder: while `fast` and `fast.next`: `slow=slow.next`, `fast=fast.next.next`. When loop exits, `slow` is the middle. For odd-length: exact middle. For even-length: second of the two middles (because `fast` exhausts before the last step)

└─ Remove Nth Node From End of List [M]

→ Two pointers with `N+1` gap: Dummy → head. Advance `fast` by `n+1` steps. Then while `fast`: `slow=slow.next`, `fast=fast.next`. Now `slow` is the predecessor of the node to delete. `slow.next = slow.next.next`

└─ Delete the Middle Node of a Linked List [M]

→ Slow/fast with `prev` pointer – land before middle: `prev = dummy`, advance `fast` two steps, `slow` one step, `prev` follows `slow`. When `fast` is `None` (or `fast.next` is `None`), `slow` is the middle node to

delete; `prev.next = slow.next`

└─ Happy Number [M]

→ Floyd's cycle detection on the implicit sequence: Define `digit_square_sum`. Run `slow/fast` until `slow == fast`. If the meeting value is 1, return `True`. Else return `False`

└─ Floyd's Cycle Detection

└─ Linked List Cycle II [M]

→ Floyd's two-phase cycle entry detection: Phase 1 – `slow` and `fast` meet after `slow` travels distance `d+c`, `fast` travels `d+c+L` (one full cycle extra), where `d` = head-to-entry, `c` = entry-to-meeting, `L` = cycle length. This implies `d = L - c = d'` (distance from meeting point back to entry). Phase 2 – `slow` resets to head, both advance 1 step at a time → they meet at the entry

└─ Find the Duplicate Number (Floyd's variant) [M]

→ Floyd's on implicit linked list defined by array values: Identical to Linked List Cycle II but operating on array indices. `slow = nums[slow]`, `fast = nums[nums[fast]]`. After meeting, reset `slow = nums[0]` (not 0, because the linked list starts from `nums[0]`)

└─ Merge / Sorting

└─ Merge Two Sorted Lists [M]

→ Dummy head + two-pointer merge: dummy → result chain. `curr` pointer builds the result. While both `l1` and `l2` non-null: attach smaller, advance it. After loop, attach the non-null remainder

└─ Merge K Sorted Lists [H]

→ Min-heap of size `k`: Initialize heap with heads of all non-null lists. While heap non-empty: pop min node, attach to result, push `node.next` if non-null

└─ Sort List [M]

→ Bottom-up merge sort on linked list: For each sublist size `size = 1, 2, 4, 8, ...`: split list into pairs of size-length sublists, merge each pair, connect results. One full pass per doubling of size, $\log n$ passes total. This is the version interviewers like when they explicitly ask for $O(1)$ extra space

└─ Insertion Sort List [M]

→ Dummy head + find-insertion-point per node: Detach each node from the original list. Walk the sorted prefix from dummy until `prev.next.val > node.val` or `prev.next` is `None`. Insert node between `prev` and `prev.next`. If the input is nearly sorted, this often behaves closer to linear time

└─ Copy / Design

└─ Copy List with Random Pointer [M]

→ Hash map original→clone, two-pass wiring: Pass 1: iterate and create `{node: ListNode(node.val)}` for all nodes. Pass 2: for each original node, set `clone.next = map[node.next]`, `clone.random = map[node.random]`. Mapping `None` → `None` keeps the wiring concise

└─ LRU Cache [M]

→ Doubly linked list + hash map: Dummy head and dummy tail eliminate all edge cases in `_remove` and `_insert_front`. On get: remove from current position, insert at front, return value. On put: if key exists, remove old; create new node, insert at front, update map; if over capacity, remove LRU (`tail.prev`), delete from map

└─ Other Manipulation

└─ Add Two Numbers [M]

→ Digit-by-digit addition with carry: $carry = 0$. While $l1$ or $l2$ or carry non-zero: sum the current digits (0 if list exhausted) + carry. New digit = $sum \% 10$, $carry = sum // 10$. Append digit node to result

Swap Nodes in Pairs [M]

→ Dummy head + iterative pair swapping: Save $first = curr.next$, $second = curr.next.next$. Then: $curr.next = second$, $first.next = second.next$, $second.next = first$. Advance $curr = first$ (first is now behind second after swap)

Intersection of Two Linked Lists [M]

→ Two pointers traversing both lists – alignment by total distance: $p1$ traverses A then B; $p2$ traverses B then A. When one reaches None, redirect to the other list's head. They travel $m+n$ and $n+m$ steps respectively – equal total. They meet at the intersection (or both reach None simultaneously = no intersection)

Rotate List (Circular List Trick) [M]

→ Make circular, find new tail, break circle: Walk to find length and tail. Connect tail to head (circular). Walk to position $n - k \% n - 1$ for new tail. $new_head = new_tail.next$. Break: $new_tail.next = None$

Remove Duplicates from Sorted List [M]

→ Single pass: skip consecutive equal nodes: Iterate $curr$. While $curr.next$ exists and $curr.next.val == curr.val$, set $curr.next = curr.next.next$. After the inner loop, advance $curr = curr.next$

Remove Duplicates from Sorted List II [M]

→ Dummy head + prev pointer skipping duplicate runs: Dummy head. $prev = dummy$. While $curr$ non-null: if $curr.next$ exists and $curr.val == curr.next.val$, record dup_val , advance $curr$ past all nodes with that value, set $prev.next = curr.next$. Else $prev = curr$. $curr = curr.next$

Partition List [M]

→ Two dummy heads – collect two sublists, then join: Null-terminate the greater chain ($greater_tail.next = None$) to avoid cycles if the original tail landed in the less chain

Flatten a Multilevel Doubly Linked List [M]

→ Walk the list. At each node with a child: push $node.next$ to stack, set $node.next = node.child$, fix prev pointers, clear $node.child$. When $node.next$ is None and stack is non-empty, pop and link

LFU Cache [H] *

→ Two hash maps + one frequency-keyed map of doubly linked lists: On get: increment freq, move key from old freq bucket to new freq bucket, update min_freq if old bucket is now empty and min_freq was that old freq. On put: if over capacity, evict from $freq_map[min_freq]$ (popitem from the front = LRU). Then insert at $freq=1$, set $min_freq = 1$

Design

LRU Cache (Doubly Linked List + Hash Map) [M] *

→ get: if key missing return -1, else move node to front and return value. put: if key exists update value and move to front; if new key and at capacity, remove the node just before the tail (LRU), then insert new node at front

queue.md

coding/data-structures/queue.md

└─ queue.md

Monotonic Deque

Sliding Window Maximum [H]

→ EDGE CASES:: For each index i – (1) evict from the back any index whose value $\leq nums[i]$ (they are dominated and can never be future maxima; using $\leq$ also drops older duplicates); (2) evict from the front if it has fallen outside the window; (3) append i ; (4) once $i \geq k-1$, the front of the deque is the answer. Each index is enqueued and dequeued at most once → $O(n)$ total. – EDGE CASES: $k = 1$ returns the original array; $k = len(nums)$ returns a single maximum

Sliding Window Minimum [M]

→ EDGE CASES:: Identical to the maximum variant; flip one comparison – evict from the back when $nums[back] \geq x$ instead of $\leq$. The front always holds the index of the current window minimum. Using $\geq$ keeps the deque short by discarding older duplicates. – EDGE CASES: $k = 1$ returns the original array; duplicate values are safe because the deque stores indices, not values

Shortest Subarray with Sum at Least K [M]

→ EDGE CASES:: Compute prefix sums. Maintain a deque of indices with strictly increasing prefix-sum values. For each j : while the front of the deque satisfies $prefix[j] - prefix[front] \geq k$, update the answer with $j - front$ and pop the front (we want the smallest valid $j - i$, so once a shorter subarray is found the front is no longer useful). Then maintain the increasing invariant by popping from the back while $prefix[back] \geq prefix[j]$, and append j . – EDGE CASES: Keep $prefix[0] = 0$ so subarrays starting at index 0 are handled naturally; if no qualifying subarray exists, return -1

Jump Game VI (DP + Sliding Window Max) [M]

→ $dp[i] = nums[i] + \max(dp[j])$ for j in $\text{range}(\max(0, i-k), i)$. Use a deque of indices in decreasing dp value order. Before computing $dp[i]$, evict indices outside the window $[i-k, i-1]$ from the front. The front of the deque is $\text{argmax } dp$ in the window

Maximum of Minimums of Every Window Size [M]

→ 1. Use monotonic stack (increasing) to compute $left[]$ (previous smaller element index, defaulting to -1) and $right[]$ (next smaller element index, defaulting to n). 2. For each i , $window_size = right[i] - left[i] - 1$; update $ans[window_size] = \max(ans[window_size], nums[i])$. 3. Suffix maximum pass: for k from $n-1$ down to 1: $ans[k] = \max(ans[k], ans[k+1])$. This handles the fact that the minimum of a larger window is $\leq$ the minimum of a smaller window – so a value that is the minimum for window size w is also a candidate for all smaller sizes

BFS / Level-order

Binary Tree Level Order Traversal [M]

→ Enqueue root. At the start of each BFS iteration snapshot $size = len(queue)$. Dequeue exactly $size$ nodes, collect their values, enqueue their children. Append the level list to results

Binary Tree Zigzag Level Order Traversal [M]

- Toggle `left_to_right` after each level. When `False`, reverse the collected level list. Children are always enqueued left-to-right; only the output list is reversed

Binary Tree Right Side View [M]

- Standard level-size snapshotting. After processing all nodes in a level, the most recently processed node value is the rightmost – append it to results

BFS Multi-Source

Rotting Oranges [M]

- Count fresh oranges. Run BFS – each time a fresh orange is infected, decrement fresh count and enqueue the new cell with time + 1. After BFS, if `fresh > 0`, return -1 (blocked cells remain). Otherwise return the last timestamp used

01 Matrix [M]

- Initialize all 0 cells with distance 0 in the queue. Initialize all 1 cells with `inf`. BFS outward – first time a 1 cell is reached sets its distance. BFS guarantees minimum distance

Walls and Gates [M]

- Enqueue all cells with value 0 (gates). BFS outward – each INF cell reached gets distance = parent's distance + 1. Walls (-1) are never enqueued or updated

Farthest Building from Land (As Far from Land as ... [M])

- Seed the queue with all 1 cells at distance 0. BFS outward filling 0 cells. Track the last distance assigned – that is the answer

Shortest Path in Binary Matrix [M]

- If start or end is 1, return -1 immediately. BFS with 8 directions. The first time (n-1, n-1) is dequeued, return the current distance + 1

Minimum Knight Moves [M]

- Reflect (x, y) to (|x|, |y|) – knight distances are symmetric. BFS from (0,0) inside a bounded box $[-2..x+2] \times [-2..y+2]$; the +2 buffer handles the small detours needed near the origin

BFS Single-Source

Word Ladder [H]

- Enqueue (beginWord, 1). For each dequeued word, generate all one-letter variants; if a variant is in the word set, enqueue it with distance + 1 and remove from the set. Return the distance when endWord is reached

Open the Lock [M]

- Each state has 8 neighbors (4 wheels × 2 directions). Mark deadends and the start as visited before BFS begins. The first time target is reached, return the current depth

Bus Routes [M]

- Build stop → [bus_indices] map. BFS starts from all buses that include source. For each bus dequeued, visit all its stops; if target is reached, return bus count. For each stop on this bus, enqueue all other buses that serve it and haven't been visited. Mark buses as visited to avoid re-boarding

Shortest Path in Grid with Obstacles Elimination [M]

- Enqueue (0, 0, k) with 0 steps. For each cell, try all 4 neighbors – if a neighbor is free, step to it; if it's an obstacle and remaining_k > 0, step to it and decrement k. Mark (r, c, k) as visited – not just (r, c)

Cheapest Flights Within K Stops [M]

- Use Bellman-Ford with exactly k+1 relaxation rounds. Maintain `prices[node]` = cheapest price to reach node using at most current_round hops. Use a copy of prices per round to avoid using within-round updates

Jump Game III [M]

- Enqueue start. For each index, compute both jump targets. If in bounds and unvisited, enqueue. Return True immediately if a target index has value 0

Topological Sort (Kahn's BFS)

Course Schedule [M]

- Build adjacency list and in-degree array. Enqueue all nodes with in-degree 0. For each dequeued node, decrement neighbors' in-degrees; enqueue any that reach 0. Count processed nodes – if count equals numCourses, return True

Course Schedule II [M]

- Append each dequeued node to order. If `len(order) == numCourses`, the graph is a DAG and order is a valid schedule. Otherwise return []

Alien Dictionary [H]

- For each adjacent word pair, find the first differing character – that gives an edge. If word A is a prefix of word B but appears after B, return "" (invalid). Run Kahn's BFS. If all characters are processed, the BFS output is the alien alphabet order

Design

Design Hit Counter [M]

- hit: append timestamp. getHits: evict front while front ≤ timestamp - 300, then return `len(deque)`. This assumes calls arrive with non-decreasing timestamps, which is the usual interview contract

Design Circular Queue [M]

- Enqueue writes to `(front + size) % cap` and increments size. Dequeue advances front by 1 (mod cap) and decrements size

Moving Average from Data Stream [M]

- Append each new value to the deque and add to sum. If the deque exceeds size, pop from the front and subtract from sum. Return `sum / len(deque)`

Number of Recent Calls [M]

- Append t. Pop from front while front < t - 3000. Return `len(deque)`

Implement Queue Using Stacks [M]

- Each element moves from push → pop exactly once, so amortized O(1) per dequeue

Implement Stack Using Queues [M]

- Push is O(n); pop and top are O(1). (Trade-off: opposite of queue-from-stacks.)

Priority Queue / Heap

Task Scheduler [M]

- Build frequency map; push all (-count,) entries to the heap. At each time tick: if the cooldown queue front is ready (available_at ≤ time), push it back onto the heap. If the heap is non-empty, pop and execute the most frequent task, push it to the cooldown queue with updated count. Otherwise, idle. Increment time

Find Median from Data Stream [M]

- addNum: push to lo (negate for max-heap), then balance by moving lo's max to hi if `lo[0] > hi[0]` or sizes diverge. Rebalance so lo

- is never smaller than hi
- Sliding Window with Queue
 - Sliding Window Median (LC 480) [M]
 - Lazy deletion: track counts of "dead" elements in each heap. When computing the median, first pop any dead elements from the heap tops. Balance rule: $\text{len}(\text{lo}) == \text{len}(\text{hi})$ or $\text{len}(\text{lo}) == \text{len}(\text{hi}) + 1$
- Scheduling
 - Task Scheduler with Cooldown (LC 621) [M]
 - Count frequencies with a Counter. max_freq = highest frequency. max_count = number of tasks that share that frequency. Formula accounts for $\text{max_freq} - 1$ complete cycles, each of length $n+1$, plus the final partial cycle of max_count tasks

segment-tree.md

coding/data-structures/segment-tree.md

- segment-tree.md
 - Common Interview Patterns
 - Range Minimum Query with Lazy Propagation [M]
 - EDGE CASES:: Range update $[\text{ul}, \text{ur}]$ with delta: if node's interval is fully covered, add delta to node.min_val and node.lazy ; otherwise push_down first, recurse on children, pull up. – Push down: apply parent.lazy to both children (add to their min_val and lazy), then clear parent's lazy . – Range min query: standard decomposition, same as sum query but return min of left/right. – EDGE CASES: Never push past a leaf; handle the empty-array case up front
 - Range Maximum Query [M]
 - EDGE CASES:: Identity for max is $-\text{inf}$ (out-of-range nodes return this). Merge: $\text{max}(\text{left_child}, \text{right_child})$. – EDGE CASES: Empty arrays should return $-\text{inf}$ for queries or be guarded explicitly, depending on the API
 - Segment Tree Applications
 - Count of Range Sum [M]
 - ALTERNATIVE:: Merge sort approach – during merge, for each right-half element $\text{prefix}[j]$, count how many left-half elements fall in $[\text{prefix}[j] - \text{upper}, \text{prefix}[j] - \text{lower}]$ using two pointers. $O(n \log n)$. – ALTERNATIVE: Coordinate compression + BIT works too, especially if you already have a Fenwick template in an interview
 - Number of Longest Increasing Subsequence [M]
 - $O(n^2)$ DP is simple. Segment tree on values (coordinate compressed) can reduce to $O(n \log n)$: tree node stores $(\text{max_length}, \text{total_count})$ for values processed so far; query $[\text{0}, \text{nums}[i]-1]$ for best, then update at $\text{nums}[i]$
 - Queue Reconstruction by Height [M]
 - This is a greedy $O(n^2)$ insertion. BIT/segment tree can optimize to $O(n \log n)$ by tracking the k-th empty slot
 - Binary Indexed Tree (Fenwick Tree)
 - Count of Smaller Numbers After Self [M]
 - Coordinate compression maps values to $1..n$. BIT query at rank-1 = count of right-side elements smaller than current. Then insert current into BIT
 - Reverse Pairs [M]
 - Coordinate compress nums (and also $\text{nums}[i]/2$ values for query). Two separate coordinate sets or unified set with both values
 - Segment Tree – Interval / Scheduling Problems
 - My Calendar I (LC 729) [M]
 - Two intervals $[\text{s1}, \text{e1}]$ and $[\text{s2}, \text{e2}]$ overlap iff $\text{s1} < \text{e2}$ and $\text{s2} < \text{e1}$. For a new booking $[\text{s}, \text{e}]$: – Find the insertion point i via bisect_left on starts. – Check left neighbor ($i-1$): does it end after s ? – Check right neighbor (i): does it start before e ? – If neither overlaps, insert
 - My Calendar II (LC 731) [M]
 - For new $[\text{s}, \text{e}]$: 1. Check if $[\text{s}, \text{e}]$ overlaps any interval in overlaps. If yes → return False. 2. Else: add intersection of $[\text{s}, \text{e}]$ with each existing booking in calendar to overlaps. 3. Add $[\text{s}, \text{e}]$ to calendar

- My Calendar III (LC 732) [M]
 - Two approaches – difference array / sorted map sweep (simpler, $O(n)$ per query) or segment tree with lazy propagation ($O(\log \text{MAX})$ per query). – HOW (difference array with SortedDict): – On book(s, e): $\text{diff}[s] += 1, \text{diff}[e] -= 1$. Sweep prefix sum, track max. $O(n)$ per call. – HOW (segment tree): – Range add [start, end-1] += 1. Query global max (root value). $O(\log \text{MAX})$ per call
- The Skyline Problem (LC 218) [M]
 - – Create events: (left, -height, right) for building starts (negative height for sort order), (right, 0, 0) for building ends.
 - Sort all events by x, then by height (starts before ends at same x – negative heights sort first).
 - Use a max-heap of (-height, right) for active buildings.
 - At each x: remove expired buildings ($\text{right} \leq \text{current x}$) from heap top. Current height = $-\text{heap}[0][0]$.
 - If height changed from previous, add [x, height] to result
- Falling Squares (LC 699) [M]
 - For each square: query max height in its interval, compute new height = query result + size, update the interval to that new height, record the global max
- Segment Tree Applications
 - Count of Smaller Numbers After Self (LC 315) – BI... [M]
 - Sort unique values to build rank map. Scan right to left: $\text{count}[i] = \text{bit.query}(\text{rank}[\text{nums}[i]] - 1)$. Then $\text{bit.update}(\text{rank}[\text{nums}[i]], 1)$
 - Range Sum Query 2D – Mutable (LC 308) [M]
 - Update (r, c) by delta: iterate $i = r+1$ by $i += i \& -i$, and for each i iterate $j = c+1$ by $j += j \& -j$, adding delta to $\text{bit}[i][j]$. Query prefix sum up to (r, c): sum over all i and j indices descending by clearing the lowest bit

stack.md

coding/data-structures/stack.md

- stack.md
 - Monotonic Stack – Next Greater/Smaller
 - Daily Temperatures [M]
 - Push index i onto the stack. When $\text{temps}[i] > \text{temps}[\text{stack}[-1]]$, pop and record $\text{result}[\text{popped}] = i - \text{popped}$. Stack holds indices of temperatures that haven't yet seen a warmer day
 - Next Greater Element II [M]
 - Initialize result = [-1] * n. Iterate 2n times. Use $i \% n$ to access elements. Only push $i \% n$ when $i < n$
 - Online Stock Span [M]
 - Pop all (p, s) where $p \leq \text{current_price}$, accumulating their spans. Push (current_price , $\text{accumulated_span} + 1$)
 - Sum of Subarray Minimums [M]
 - Use monotonic increasing stack twice (or once with careful boundary tracking). Left boundary: strict < comparison; right boundary: $\leq$ to avoid double-counting equal elements
 - Monotonic Stack – Histogram / Rectangle
 - Largest Rectangle in Histogram [M]
 - Append sentinel 0 to flush the stack. On pop, height = $\text{heights}[\text{popped}]$. Width = $i - \text{stack}[-1] - 1$ if stack is non-empty, else i (the bar is the global minimum so far)
 - Maximal Rectangle [M]
 - $\text{heights}[j] = \text{heights}[j] + 1$ if $\text{matrix}[\text{row}][j] == '1'$ else 0. Run $\text{largest_rectangle_area}(\text{heights})$ for each row
 - Trapping Rain Water (stack approach) [H]
 - Push indices onto a decreasing stack. On pop (taller bar arrived), compute bounded water above the popped bar
 - Valid Parentheses / Nesting
 - Valid Parentheses [M]
 - Map ')' → '(', etc. If stack is empty when closing bracket arrives, or top doesn't match, return False. Valid iff stack is empty at end
 - Decode String [M]
 - Parse digits to get k. On [: push (current_string , k), reset both. On]: pop (prev_str , k), set $\text{current} = \text{prev_str} + \text{current} * k$. Characters append to current
 - Remove All Adjacent Duplicates in String [M]
 - Linear scan. Maintain stack. At each character, cancel with top if equal. Analogous to bracket matching
 - Remove K Digits [M]
 - Build the stack left to right. Pop larger elements while $k > 0$. If k still > 0 after the loop, trim last k digits from the stack (which is already sorted ascending). Strip leading zeros
 - Stack Design
 - Min Stack [M]
 - On push, min_stack pushes $\min(\text{val}, \text{min_stack}[-1])$. On pop, both stacks pop. get_min returns $\text{min_stack}[-1]$
 - Maximum Frequency Stack [M]
 - Push: increment $\text{freq}[\text{val}]$, append val to $\text{group}[\text{freq}[\text{val}]]$, update max_freq. Pop: take from $\text{group}[\text{max_freq}]$, decrement $\text{freq}[\text{val}]$, decrement max_freq if the bucket is now empty

- Queue from Stacks
 - Implement Queue using Stacks [M]
 - push appends to `in_stack`. `pop/peek`: if `out_stack` is empty, move all of `in_stack` to `out_stack` (reversal). Then operate on `out_stack`. Amortized $O(1)$
 - Validate Stack Sequences [M]
 - Maintain pointer `j` into popped. After pushing `pushed[i]`, pop while `stack` and `stack[-1] == popped[j]`, incrementing `j`. Valid iff `stack` is empty at end
- Expression Evaluation
 - Evaluate Reverse Polish Notation [M]
 - Pop `b` then `a` (order matters for `-` and `/`). Apply operator. Push result. Division truncates toward zero: `int(a / b)` not `a // b` (handles negatives)
 - Basic Calculator I [M]
 - Parse multi-digit numbers. Accumulate into result using current sign. Handle `(` and `)` for scope management
 - Basic Calculator II [M]
 - Parse number, apply `prev_op` with `stack`. Default `prev_op = '+'`. Final answer = sum of `stack`
 - Basic Calculator III [M]
 - Implement a helper that processes until end or `)`. Inside, use the stack-based approach from Calculator II. On `(`, recurse for the inner expression
- Simulation / Other
 - Exclusive Time of Functions [M]
 - Parse each log. On `"start"`: if `stack` non-empty, add elapsed time to top's exclusive time. Push current. On `"end"`: pop, compute time, add to result. Update `prev_time = end + 1`
 - Simplify Path [M]
 - Split on `/`. For each part: skip empty strings and `.`; pop on `..` (if `stack` non-empty); push otherwise. Join with `/` and prepend `/`
 - Baseball Game [M]
 - Parse each op. Integer: push. `+`: push `stack[-1] + stack[-2]`. `D`: push `stack[-1] * 2`. `C`: pop. Sum the `stack` at the end
 - Asteroid Collision [M]
 - While collision conditions hold: if top is smaller, pop (top destroyed, current continues); if equal, pop and break (both destroyed); if top is larger, break (current destroyed, don't append). Use `while...else` to append only if current survived
- Parentheses – Score and Repair
 - Score of Parentheses (LC 856) [M]
 - Start with `[0]`. On `(`: append `0`. On `)`: `v = stack.pop()`; `stack[-1] += max(2*v, 1)`. Return `stack[0]`
 - Minimum Add to Make Parentheses Valid (LC 921) [M]
 - Final answer = open + close
 - Check if Word is Valid After Substitutions (LC 1003) [M]
 - After each push check if `stack[-3:] == ['a','b','c']` and pop three. Valid iff `stack` is empty at end
- Monotonic Stack – Arrays and Sequences
 - 132 Pattern (LC 456) [M]
 - `third = -inf`. For each element right to left: if `num < third`, return `True`. While `stack` and `stack[-1] < num`, set `third = stack.pop()`. Push `num`
 - Car Fleet (LC 853) [M]
 - Iterate sorted arrival times. Push if `> stack[-1]` (or `stack` empty). `Stack size = number of fleets`
 - Maximum Width Ramp (LC 962) [M]
 - Build decreasing stack in one pass. Reverse scan: greedily pop all valid left endpoints
 - Number of Visible People in a Queue (LC 1944) [M]
 - Process right to left. For each person, pop from the decreasing `stack` while `top < current height`, incrementing count. Add 1 if `stack` non-empty (blocked by a taller person). Push current height
 - Buildings With an Ocean View (LC 1762) [M]
 - Walk right to left. If `heights[i] > max_right`, append `i` to results and update `max_right`. Reverse the results before returning (indices must be in ascending order)
- Iterative Tree Traversal
 - Flatten Binary Tree to Linked List (LC 114 – iter... [M]
 - Push root. While `stack`: pop node, if `node.right` exists push it, if `node.left` exists push it. Set `node.right = stack[-1]` if `stack` else `None`, `node.left = None`
 - Path Sum II (LC 113 – iterative DFS) [M]
 - Push (`root`, `target`, `[]`). On each pop: if leaf and `remaining == 0`, add copy of path to results. Push right child, then left child (left processed first) with updated `remaining` and path
- Monotonic Stack – Advanced
 - Sum of Subarray Minimums (LC 907) [H]
 - In one pass, use the `stack` to track unresolved indices. When `nums[i]` is smaller than the `stack` top, pop and compute the contribution of the popped element with `i` as its right boundary. Left boundary comes from the new `stack` top (or `-1` if empty)

string.md

coding/data-structures/string.md

└─ string.md

└─ Frequency Map / Anagram

└─ Valid Anagram [E]

→ Build Counter(s) and Counter(t). Return Counter(s) == Counter(t). For pure lowercase ASCII, use [0]*26 and compare arrays – same asymptotic cost, better constant

└─ Group Anagrams [M]

→ For each word, compute key = tuple(freq_array) or key = "".join(sorted(word)). Append word to groups[key]. Return list(groups.values())

└─ Find All Anagrams in a String [M]

→ Build p_freq. Maintain w_freq for the window. Add the incoming character on the right; evict the outgoing character on the left when the window exceeds len(p). If arrays match, record the left index. Use a matches counter to avoid O(26) comparison: track how many of the 26 buckets currently match between w_freq and p_freq

└─ Two Pointers – Palindrome

└─ Valid Palindrome [M]

→ While l < r: skip l while not alphanumeric, skip r while not alphanumeric. If s[l].lower() != s[r].lower(), return False. Advance both pointers inward

└─ Longest Palindromic Substring [M]

→ expand(l, r) expands while s[l] == s[r] and indices are in bounds, returning the palindrome substring. For each index i, try both expand(i, i) (odd length) and expand(i, i+1) (even length). Track the longest result

└─ Palindromic Substrings (Count All Palindromic Sub... [M]

→ For each of the 2n - 1 centers, expand while s[l] == s[r]. Each valid (l, r) pair is one palindromic substring – increment count by 1 each step

└─ Sliding Window

└─ Longest Substring Without Repeating Characters [M]

→ For each right: if s[right] is in the set, remove s[left] and advance left until the duplicate is gone. Then add s[right] and update max

└─ Minimum Window Substring [H]

→ Expand right: add s[right] to have; if have[c] == need[c], increment formed. When formed == len(need) (window valid): record min window, shrink from left – decrement have[s[left]]; if it drops below need[s[left]], decrement formed. Repeat shrinking until no longer valid

└─ Longest Repeating Character Replacement [M]

→ Expand right: update count[s[right]] and max_freq. If (window_size - max_freq) > k, slide left by 1 – don't shrink, just slide. The window grows when a valid longer window is found

└─ Prefix Sum on Characters

└─ Prefix Sum on Characters Pattern [M]

→ Prefix count per char; window validity = freq map compare

└─ Number of Substrings Containing All Three Characters [M]

→ Maintain last = [-1, -1, -1] for a/b/c. For each r, update last[ord(s[r]) - ord('a')] = r. Add min(last) + 1 to answer

(clamped to 0 when any char unseen)

└─ Count Vowel Substrings of a Word [M]

→ atMost(k): l = 0, freq = {}. For each r: if s[r] not a vowel, reset window (l = r+1, freq = {}). Else update freq[s[r]]. While len(freq) > k, shrink l. Add r - l + 1. Answer = atMost(5) - atMost(4)

└─ Binary Subarrays With Sum [M]

→ seen = {0: 1}, running = count = 0. For each x: running += x, count += seen.get(running - goal, 0), seen[running] = seen.get(running, 0) + 1

└─ Subarray Sum Equals K (character version) [M]

→ For each character ch: running += (1 if ch == target else 0). count += prefix_count[running - k]. prefix_count[running] += 1

└─ Encoding / Hashing

└─ Encode and Decode Strings [M]

→ Encode: for each string, emit str(len(s)) + '#' + s. Decode: read digits up to # to get length L; read exactly L characters as the next string; advance pointer past them; repeat

└─ String Hashing (Duplicate Substring Detection) [M]

→ Compute hash(s[0..L-1]). For each subsequent position, update the hash by the sliding formula. Store hashes in a set. On collision, verify with string comparison to rule out false positives

└─ Parsing / Simulation

└─ String to Integer (atoi) [M]

→ Advance i past spaces. Read sign. Accumulate result = result * 10 + digit for each digit character. Before adding, check for overflow: if result > (INT_MAX - digit) // 10, clamp and return. Return sign * result

└─ Longest Common Prefix [M]

→ Take strs[0] as the reference. For each character index i in strs[0], check every other string at position i. If any string is shorter than i or differs at i, return strs[0][:i]

└─ Two Pointers – Reverse / Subsequence

└─ Reverse String [M]

→ while l < r: s[l], s[r] = s[r], s[l]; l += 1; r -= 1

└─ Is Subsequence [M]

→ Single pass through t. If i reaches len(s), all characters matched – return True

└─ Parsing / Simulation (Extended)

└─ Decode String [M]

→ Two stacks (count_stack, string_stack) or a single character stack. On digit: accumulate k. On [: push current string and k onto stacks, reset. On]: pop string and k, append k * current_string to the popped prefix

└─ Compare Version Numbers [M]

→ v1 = list(map(int, version1.split('.'))). Pad shorter list with zeros. Compare element by element

└─ Integer to English Words [M]

→ Process groups of 3 digits from largest to smallest. For each non-zero group, call helper(group) and append the appropriate suffix. Special-case 0

└─ Pattern Matching

└─ Implement strStr (KMP) [M]

→ Build LPS: two pointers len_ = 0, i = 1. If needle[i] ==


```

        needle[len_], lps[i] = len_ + 1; i++; len_++. Else if len_ > 0,
        len_ = lps[len_-1]. Else lps[i] = 0; i++. Search: advance i
        (haystack), j (needle); on mismatch use j = lps[j-1]; when j ==
        len(needle) record match

```

Repeated Substring Pattern [M]

```

→ return s in (s + s)[1:-1]. Alternatively, use KMP: build LPS
  array for s; if lps[-1] > 0 and len(s) % (len(s) - lps[-1]) == 0,
  the pattern length is len(s) - lps[-1]

```

Longest Happy Prefix [M]

```

→ Standard LPS build (same as in Implement strStr). Return
  s[:lps[-1]]

```

String Manipulation

String Compression [M]

```

→ Maintain read and write pointers. For each run, count length and
  emit the character plus its decimal digits

```

Zigzag Conversion (LC 6) [M]

```

→ Track current_row (starts at 0) and direction (+1 going down, -1
  going up). Flip direction when current_row == 0 or current_row ==
  numRows - 1. Final answer = concatenation of all row strings

```

Valid Palindrome II (LC 680) [M]

```

→ Helper is_palindrome(l, r) checks s[l..r]. Main: walk until
  mismatch, then return is_palindrome(left+1, right) or
  is_palindrome(left, right-1)

```

Reverse Words in a String (LC 151) [M]

```

→ words = s.split() → words.reverse() → return ' '.join(words)

```

tree.md

coding/data-structures/tree.md

tree.md

DFS Traversal

Binary Tree Inorder Traversal [M]

```

→ while curr or stack: inner while curr pushes all lefts; curr =
  stack.pop() processes node, appends value; curr = curr.right to
  explore right subtree

```

Invert Binary Tree [M]

```

→ Post-order recursive swap: Base case not root → None. Recurse
  left and right, then swap: root.left, root.right =
  invertTree(root.right), invertTree(root.left). Pre-order also
  works since swapping is an O(1) local operation

```

Symmetric Tree [M]

```

→ Recursive mirror(l, r) – outer and inner pair matching: Base
  cases: both None → True (symmetric absence), exactly one None →
  False. Otherwise: l.val == r.val and mirror(l.left, r.right) and
  mirror(l.right, r.left)

```

Maximum Depth of Binary Tree [M]

```

→ Post-order recursive max height: Base case not root → 0;
  otherwise 1 + max(maxDepth(left), maxDepth(right))

```

Minimum Depth of Binary Tree [M]

```

→ Base case: not root → 0. Then check left/right nullity before
  min

```

Path Sum [M]

```

→ DFS subtracting current value – check at leaf: The leaf check is
  critical – only return True at nodes where not left and not right
  (both children null), not at any node where partial sum matches

```

Path Sum II [M]

```

→ At leaf (not left and not right) and remaining == 0:
  result.append(list(path)) (copy! not reference). Then path.pop()
  on return regardless of whether this was a leaf

```

Sum Root to Leaf Numbers [M]

```

→ dfs(node, curr_num) – curr_num = curr_num * 10 + node.val; at
  leaf return curr_num; otherwise return dfs(left, curr_num) +
  dfs(right, curr_num)

```

Count Complete Tree Nodes [M]

```

→ height(node) follows .left pointers only. If left_h == right_h,
  return 2^left_h + countNodes(root.right); else return 2^right_h +
  countNodes(root.left)

```

Count Good Nodes in Binary Tree [M]

```

→ DFS with propagated path_max: Update path_max = max(path_max,
  node.val) before recursing; root is always good

```

Same Tree [M]

```

→ Base cases: both None → True; exactly one None → False; p.val !=
  q.val → False. Recurse: isSameTree(p.left, q.left) and
  isSameTree(p.right, q.right)

```

Subtree of Another Tree [M]

```

→ isSubtree(root, subRoot) – if not root: False; if
  isSameTree(root, subRoot): True; else recurse left and right

```

Flatten Binary Tree to Linked List [M]

```

→ Morris-style in-place threading – find inorder predecessor: For
  each curr with a left child: find rightmost node in left subtree

```

- (prev), wire prev.right = curr.right, move curr.right = curr.left, null curr.left. Advance curr = curr.right
- Step-By-Step Directions From a Binary Tree Node t... [M]
 - DFS to root→start and root→dest paths, strip common prefix: DFS to record L/R path from root to each target node, then strip common prefix
- Level Order BFS
 - Binary Tree Level Order Traversal [M]
 - BFS with level-size snapshot: Inner loop runs exactly level_size times; enqueue children during inner loop; append collected level after inner loop
 - Binary Tree Zigzag Level Order Traversal [M]
 - left_to_right = True initially; after collecting each level: if not left_to_right, level.reverse(); then left_to_right = not left_to_right
 - Binary Tree Right Side View [M]
 - BFS – capture last node per level: Snapshot level size; run inner loop; append value only when i == level_size - 1
 - Populating Next Right Pointers in Each Node [M]
 - O(1) space – walk current level to connect next level: Start with leftmost = root; inner loop walks the current level using head.next; outer loop descends via leftmost = leftmost.left
 - Vertical Order Traversal of a Binary Tree [M]
 - DFS collect (col, row, val) tuples, sort, group: DFS assigning (row, col) to each node, collect all tuples, sort, then group by column
 - All Nodes Distance K in Binary Tree [M]
 - Build undirected graph, BFS k steps from target: Build graph first, BFS second, collect nodes at distance == k
- Lowest Common Ancestor
 - Lowest Common Ancestor of a Binary Tree [M]
 - Post-order recursion – converge at split node: if left and right: return root; return left or right
 - Lowest Common Ancestor of a BST [M]
 - Iterative BST-guided descent – O(h) instead of O(n): If both p, q < root, go left. If both > root, go right. Otherwise return root
- Tree DP
 - Diameter of Binary Tree [M]
 - Post-order height DFS with global diameter update: Return 1 + max(left, right) upward; update best[0] with l + r at each node
 - Binary Tree Maximum Path Sum [H]
 - Post-order gain DFS – clamp negatives to 0: gain(node) = node.val + max(l, r) returned upward; global update = node.val + l + r (where l and r already have negatives clamped to 0)
 - House Robber III [M]
 - Post-order DP returning (rob, skip) pair: rob = node.val + l_skip + r_skip; skip = max(l_rob, l_skip) + max(r_rob, r_skip). Post-order: compute children first
 - Binary Tree Cameras [M]
 - Greedy post-order – delay cameras upward: Null nodes return 1 (trivially covered). If any child returns 0, place a camera here (return 2, increment count). If any child has a camera (returns 2), this node is covered (return 1). Otherwise return 0 (push responsibility to parent)
 - Path Sum III [M]

- DFS with prefix sum hash map + backtracking: DFS – add node.val to running sum, query map for running_sum – targetSum, increment map, recurse children, then decrement map on backtrack (critical: prevents prefix sum from leaking into sibling branches)
- BST Operations
 - Validate Binary Search Tree [M]
 - Recursive range validation – propagate (lo, hi) bounds: validate(node, lo, hi) – fail if not lo < node.val < hi, else recurse with tightened bounds
 - Kth Smallest Element in a BST [M]
 - Iterative inorder with early exit at k: while stack or root: push all left children, pop, decrement k, if k == 0 return val, else advance to right child
 - Insert and Delete in BST [M]
 - Delete's two-child case: find inorder successor (leftmost in right subtree), copy its value to current node, then delete successor from right subtree
 - Range Sum of BST [M]
 - DFS with BST pruning – skip entire subtrees: Only recurse left if node.val > low; only recurse right if node.val < high
 - Recover Binary Search Tree [M]
 - Inorder DFS – detect inversion pair(s), swap values: One or two inversion sites. Always: first = prev at first inversion; second = curr at each inversion (covers both one and two inversion cases)
- Construction / Serialization
 - Serialize and Deserialize Binary Tree [H]
 - Preorder DFS with null markers – iterator-based deserialization: Serialize: DFS pre-order appending values or #. Deserialize: iterate tokens; # → return None; otherwise create node, recurse left, recurse right. Use an iterator to advance position across recursive calls
 - Construct Binary Tree from Preorder and Inorder T... [M]
 - Preorder index advance + inorder hash map for O(1) root lookup: build(in_left, in_right) – take preorder[pre_idx] as root, find its inorder position mid, build left subtree with in_left..mid-1, right subtree with mid+1..in_right
 - Construct Binary Tree from Inorder and Postorder ... [M]
 - build(in_left, in_right) – take postorder[post_idx] as root, decrement index, find root in inorder hash map as mid, build right subtree (mid+1, in_right) FIRST, then left (in_left, mid-1)
- Special Tree Problems
 - Count Complete Tree Nodes (LC 222) [M]
 - left_height = length of left spine. right_height = length of right spine. If equal, return (1 << left_height) - 1. Else return 1 + count(root.left) + count(root.right)
 - Binary Tree Cameras (LC 968) [H]
 - For each node: if either child is uncovered (state 0), place a camera here (state 1, increment count). If either child has a camera (state 1), this node is covered (state 2). Otherwise (both children covered without cameras), return state 0 – let the parent handle coverage. After DFS, if root returns state 0, place one more camera
 - Recover Binary Search Tree (LC 99) [M]
 - After traversal, swap first_bad.val and second_bad.val

trie.md

coding/data-structures/trie.md

└ trie.md

└ Core Trie Implementation

└ Implement Trie (Prefix Tree) [M]

- EDGE CASES:: - insert: walk the tree character by character, creating nodes as needed, set is_end = True at the last character.
- search: walk the tree; return False if any character is missing; return node.is_end at the end - this distinguishes "apple" (exact) from "app" (only prefix). - startsWith: same walk as search but return True after the walk completes regardless of is_end.
- EDGE CASES: The empty string is valid only if you explicitly mark the root as terminal; duplicate inserts are idempotent with a boolean trie; deletes must prune only dead branches so shared prefixes stay intact

└ Autocomplete / Prefix Search

└ Design Search Autocomplete System [M]

- On insert(sentence, freq): walk each character; at each node update node.counts[sentence] += freq. On input(c): if '#', save the current input with frequency 1 (updating all ancestor nodes), reset state. Otherwise, advance curr_node by one character and return the top 3 sentences from curr_node.counts, ordered by (-frequency, sentence)

└ Map Sum Pairs [M]

- On insert(key, val): if key already exists, compute delta = val - old_val to avoid double-counting. Walk each character of key; at each node add delta to node.prefix_sum. On sum(prefix): walk to the prefix node and return node.prefix_sum

└ Longest Word in Dictionary [M]

- Insert all words. BFS from root - only enqueue child nodes where is_end = True. At each node, track the current word. Update the answer when a longer (or lexicographically smaller tie) word is found

└ Replace Words [M]

- Insert all roots into the trie. For each word in the sentence, walk the trie; if is_end is reached at depth d, replace the word with word[:d]. If the walk exits without finding a root, keep the original word

└ Trie + Backtracking

└ Word Search II [H]

- Build trie, storing the actual word string at is_end nodes. DFS: if ch not in node.children, return immediately. If node.word is set, record and clear it (deduplication). Mark cell as '#' during DFS; restore on backtrack. After DFS, prune dead trie nodes (del node.children[ch] when subtree is empty) to avoid re-exploring exhausted branches

└ Word Squares [M]

- Build trie; at each node store all words whose prefix passes through it (node.words). Backtrack row by row: at row k, the required prefix = square[0][k] + square[1][k] + ... + square[k-1][k] (k-th column of already-placed words). Look up all words with that prefix in the trie. Try each as square[k]; recurse to row k+1. Base case: len(square) == word_length → record result

└ XOR Trie

└ Maximum XOR of Two Numbers in an Array [M]

- insert(num): for bits 31 → 0, compute b = (num >> bit) & 1, follow or create the b branch. max_xor(num): for each bit, try to go to 1 - b (opposite = XOR bit = 1); if that branch exists, take it and add 1 << bit to the XOR; otherwise take the b branch. Build the trie with all numbers, then query each number for its best XOR pair

└ Miscellaneous

└ Number of Distinct Substrings in a String [M]

- Start with an empty trie. For each suffix, insert it - count the number of new nodes created. The total count is the number of distinct substrings. (Equivalently: total nodes in the trie minus the root.)

└ Core Trie Operations (Extended)

└ Add and Search Word [M]

- addWord: standard trie insert. search(word, node, i): if i == len(word) return node.is_end. If word[i] == '.', recurse into every child and return True if any succeeds. Otherwise follow the exact child as usual

└ Prefix Problems

└ Search Suggestions System [M]

- Sort products. Insert each: at every node on the path, append the word to node.suggestions if len < 3. Query: walk the search word prefix character by character; at each step return node.suggestions (or [] if the branch doesn't exist, and stay dead for subsequent characters)

└ Prefix and Suffix Search [M]

- For each word words[i], insert every suffix of that word followed by '#' and the full word. Query f(pref, suff) walks suff + '#' + pref in the trie and returns the stored max index, or -1 if the path is missing

└ Count Words With Given Prefix [M]

- Insert each word, incrementing node.count at every node on the path. Query: walk pref; if the walk succeeds, return node.count. If any character is missing, return 0

└ Implement Magic Dictionary [M]

- dfs(node, word, i, diff): base case i == len(word) → return node.is_end and diff == 1. At each step: for exact char, recurse with same diff. For all other children, recurse with diff + 1 (only if diff == 0). Return True if any branch returns True

└ XOR Trie (Extended)

└ Maximum XOR With an Element From Array [M]

- Sort nums. Sort queries by mi (keep original index for output). Two pointers: pointer j into nums. For each query (xi, mi), advance j while nums[j] <= mi, inserting into trie. If trie is empty (no num ≤ mi), answer is -1. Otherwise query max_xor(xi)

└ Count Pairs With XOR in a Range [M]

- count_leq(num, limit): walk bit by bit from MSB. At each bit b of num and l of limit: if l == 1, all numbers with XOR bit 0 at this position contribute (they have XOR < current prefix) - add node.children[b].size if it exists, then continue down the 1-b branch to keep XOR bit = 1. If l == 0, must go down b branch (XOR bit = 0)

└ Bitwise Trie / Other

Design File System [M]

→ createPath(path, value): split path by /, ignore leading empty string. Walk all but the last component – if any is missing, return False. At the last component, if the node already exists and was explicitly created (value != -1), return False; else create it with the value. get(path): walk all components; return node's value or -1 if path doesn't exist

Stream of Characters [M]

→ Build trie of reversed words. Keep active: list[TrieNode]. On query(c): new_active = []; for each node in active, if c in node.children, add child to new_active. Also check root for c (new suffix starting). If any node in new_active has is_end = True, return True

Palindrome Pairs [M]

→ At each trie node, store a list of word indices whose reversed word ends here but the trie path continues – these are "prefix palindrome" candidates. Walk trie with w; if node.word_idx != -1 and the remaining w[i:] is a palindrome, record (word_idx, i_word). After exhausting w, collect from node.palindrome_suffixes (stored during build)

Design Search Autocomplete System (Trie + DFS Var... [M]

→ insert(sentence, freq): standard trie, set node.freq = freq at terminal. query(prefix): walk to prefix node. DFS collecting (freq, built_string) for all is_end nodes. Sort and return top 3

Suffix Trie / Advanced

Implement Trie II (Count Operations) [M]

→ insert: walk, incrementing node.pass_count at every node, node.end_count at terminal. countWordsStartingWith(prefix): walk to prefix node, return node.pass_count. countWordsEqualTo(word): walk to terminal, return node.end_count. erase(word): walk, decrementing node.pass_count; decrement node.end_count at terminal. Optionally prune nodes where pass_count == 0

Shortest Unique Prefix for Every Word [M]

→ Insert all words, incrementing node.pass_count at each node. For each word, walk character by character; the first node with pass_count == 1 marks the end of the shortest unique prefix

Maximum XOR of Two Numbers – Prefix Hash Approach [M]

→ max_xor = 0. For bit b from 31 to 0: mask = max_xor | (1 << b). Compute prefix set {num & mask for num in nums}. Try candidate = max_xor | (1 << b). If any two prefixes a, b in the set satisfy a ^ b == candidate (i.e., candidate ^ a is in the set), then max_xor = candidate. Else leave max_xor unchanged (this bit stays 0)

Trie Applications

Replace Words (LC 648) [M]

→ Insert all roots. For each sentence word, traverse the trie; if a node has is_end = True, return the prefix built so far. If traversal ends without a match, keep the original word

Palindrome Pairs (LC 336) [M]

→ For each word at index i, split at every position k (0 to len): if word[:k] is a palindrome and reverse(word[k:]) is in the map (and not i), add (map[reverse], i). If word[k:] is a palindrome and reverse(word[:k]) is in the map, add (i, map[reverse]). Avoid duplicates by only doing suffix-palindrome check for k > 0

Word Squares (LC 425) [M]

→ Insert all words into the trie, at each node also storing the

list of words with that prefix. Backtrack with build(step, square): if step == n, save the square. Otherwise extract prefix from the current partial square, look up matching words in the trie, and recurse